DSA 495 601 Project Milestone 3

Thomas Grover

- Set-up
- Step 1: Outcome Preparation & Checks
- Step 3: Data Processing for Predictive Analytics
- Step 4: Review and check your results
- Step 5: Save your processed training and testing files

For Milestone 3, you will prepare your data for predictive analytics. This R notebook provides a review of the data preparation concepts we discuss in class and helps you implement them. Please read through the entire document closely. You may not need to carry out all steps (e.g. if you do not have missing values, skip over the step that addresses this issue), but you can still learn by reading the notes and reviewing the code.

Your goals are (1) to create *new versions* of variables that need cleaning or processing (e.g. handling missing values, turning text categorical variables into numeric dummies, scaling) and (2) create *new variables* that summarize other variables in new ways (feature engineering).

To help you identify places where you need to edit the code to match your dataset, you can look for “@TODO” in comments. Also, please delete examples discussed in the text and add your own explanations (documentation or rationale) for decision-making for your own dataset and project. Finally, please answer the questions (also indicated with “@TODO”).

After you have edited the code as needed for your data and project goals and run all code chunks successfully, knit your completed file to a HTML. Download the HTML to your computer and then print or save it to a PDF file with the following naming convention: course_section_semester_assignment_unityID.pdf. For example, DSC495_601_FA25_Milestone3_keporte2.pdf.

Set-up

Load the necessary packages

No edits are needed in the following chunk. Just run it as is.

```
.libPaths(c("/share/dsa495601f25/installed_libraries", .libPaths()))
library(dplyr)
library(tidyr)
library(skimr)
library(janitor)
library(rsample)
library(fastDummies)
```

Read in your data

Here you can copy your code from Milestone 2.

```
data_file <- paste0(params$data_path,"bank-full.csv")
df <- read.csv(data_file, sep = ";") %>% janitor::clean_names()

dim(df)
```

```
## [1] 45211    17
```

```
head(df)
```

	age <int>	job <chr>	marital <chr>	education <chr>	default <chr>	balance <int>	housing <chr>	loan <chr>	contact <chr>
1	58	management	married	tertiary	no	2143	yes	no	unknown
2	44	technician	single	secondary	no	29	yes	no	unknown
3	33	entrepreneur	married	secondary	no	2	yes	yes	unknown
4	47	blue-collar	married	unknown	no	1506	yes	no	unknown
5	33	unknown	single	unknown	no	1	no	no	unknown
6	35	management	married	tertiary	no	231	yes	no	unknown

6 rows | 1-10 of 18 columns

Summarize your data

This will help you get you starting in identifying what variables need data cleaning and processing. But you may need to add additional summaries or descriptive analyses.

```
# Use skimr package to get summaries of all variables
skim(df)
```

Data summary

Name	df
Number of rows	45211
Number of columns	17
_____	
Column type frequency:	
character	10
numeric	7
_____	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
job	0	1	6	13	0	12	0
marital	0	1	6	8	0	3	0
education	0	1	7	9	0	4	0
default	0	1	2	3	0	2	0
housing	0	1	2	3	0	2	0
loan	0	1	2	3	0	2	0
contact	0	1	7	9	0	3	0
month	0	1	3	3	0	12	0
poutcome	0	1	5	7	0	4	0

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
y	0	1	2	3	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
age	0	1	40.94	10.62	18	33	39	48	95	
balance	0	1	1362.27	3044.77	-8019	72	448	1428	102127	
day	0	1	15.81	8.32	1	8	16	21	31	
duration	0	1	258.16	257.53	0	103	180	319	4918	
campaign	0	1	2.76	3.10	1	1	2	3	63	
pdays	0	1	40.20	100.13	-1	-1	-1	-1	871	
previous	0	1	0.58	2.30	0	0	0	0	275	

Step 1: Outcome Preparation & Checks

First, in the empty code chunk below, create your *binary* outcome variable, if it does not already exist. You may reuse code from **Milestone 2**, but consider modifying it if needed.

Question 1: Describe your outcome to be predicted and what each value (0 and 1) means.

The outcome to be predicted is whether or not a customer will open a term deposit. This is represented by a binary variable derived from the given categorical variable, y. The original variable y takes values “Yes” and “No”. For the purpose of the model I am converting these into binary outcomes, where (1 = “Yes”) and (0 = “No”).

```
# Create binary (1 or 0) version of your outcome
df <- df %>%
  mutate(y_binary = if_else(y == "yes", 1, 0))

# Check prevalence (percent of rows where the outcome is 1)
mean(df$y_binary, na.rm = TRUE)
```

```
## [1] 0.1169848
```

Step 2: Data Partitioning

In this step, you’ll divide your dataset into two parts:

1. **Training/Validation set**
2. **Testing set**

Partitioning your data *before* any further preparation is crucial to prevent **data leakage** — where information from the test set unintentionally influences model training or feature engineering. To avoid this, all remaining data preparation steps will be performed *separately* on the training/validation and testing sets.

Question 2: Did you use random splitting or time-based splitting? Why? Did you stratify your split? If so, by which variable(s), and why?

I used random splitting for my dataset because of the imbalance in my target variable y. I wanted to ensure proportional representation in both training and testing splits. Since the variable y only has about 12 percent positive cases, and this is our target variable, it is important to ensure proportional representation. Time-based splitting would be interesting to look at if I was filtering for customers at a specific point in the campaign process, but since I am not specifying this I decided to use a stratified random split.

Code Example: Splitting and Saving Your Data

Use the following code to split your dataset:

```
# For random splitting
split <- initial_split(df, prop = 0.70, strata = y_binary)
trainDat <- training(split)
testDat  <- testing(split)

# Verify train/test data split
# Check number of rows in training dataset
nrow(trainDat)
```

```
## [1] 31647
```

```
# Check number of rows in test dataset
nrow(testDat)
```

```
## [1] 13564
```

```
# Verify that train + test equals original dataset (should be TRUE)
nrow(trainDat) + nrow(testDat) == nrow(df)
```

```
## [1] TRUE
```

```
# Calculate proportion of data used for training
nrow(trainDat) / nrow(df)
```

```
## [1] 0.6999845
```

Step 3: Data Processing for Predictive Analytics

After creating your outcome variable and partitioning your data into training and testing sets, data processing is crucial for model performance. This section will walk you through the essential steps.

(a) Handling Missing Values

When building predictive models, it's important to handle missing values carefully. Missingness itself can contain useful information — so we don't always want to just drop or fill in the missing values without keeping track of them.

Question 3: Do you have missing values? If so, which option(s) did you use to handle missing values (dummy variable adjustment, binning, or "Missing" category)? Why did you make the choices you did? Do you think missingness itself might be predictive in your dataset? Explain briefly.

The Bank-Marketing dataset does not contain any true missing values. As a result, I did not need to utilize the common methods for dealing with missing values. However, there are variables in the dataset that can take the value "unknown". These include education, contact (contact communication type), and poutcome (outcome of the previous campaign). This basically represents missing information.

I thought it was better to treat this "Unknown" as its own category and deal with it later using One-Hot Encoding rather than treating it as missing. I think this "missingness" itself might be predictive, particularly if the missingness relates to contact issues or lack of records. This could reflect customers that had little contact with the bank. I think there could be information in this data, so I thought it best to preserve it.

(b) One-Hot Encoding (Dummy Variables) for Categorical Variables

Some machine learning models — like `glmnet` (for lasso, ridge), `keras` (for neural networks), or support vector machines — **only work with numeric data**. This means we need to turn categorical variables into numbers.

The most common way to do this is called **one-hot encoding** (or **dummy coding**). This creates a new column for each category in your variable, with values of `1` (yes) or `0` (no).

As noted above, this procedure will also deal with missing values in categorical variables because it will create a missing value for NA or other indicator of missing.

✅ Dummy Coding with `fastDummies::dummy_cols()`

Let's say you want to create dummy variables for:

- `"school_type"`
- `"testscore_cat"`

Here's the code:

```
# Specify columns of categorical variables you want to one hot code
cat_columns = c("job", "marital", "education", "default", "housing", "loan", "contact", "month", "poutcome")

trainDat <- fastDummies::dummy_cols(
  trainDat,
  select_columns = cat_columns,
  remove_selected_columns = TRUE
)

testDat <- fastDummies::dummy_cols(
  testDat,
  select_columns = cat_columns,
  remove_selected_columns = TRUE
)
```

Question 4: Which variables did you one-hot encode? Why?

I one-hot encoded every categorical variable other than the target. I did this because I needed to quantify the categorical variables, and one-hot encoding made a lot of sense for categories like marital status, job, and contact type that contained multiple categories. I also one-hot encoded every variable that could take the value "Unknown", such as education and poutcome. I did this to represent unknown rather than treat it as missing. This helps get as much predictive meaning as possible from the data and avoid leakage.

(c) Scaling

Often, variables used in machine learning models have different ranges. Without scaling, variables with larger absolute values will dominate the prediction outcome regardless of their actual importance. To give each feature a fair opportunity to contribute to the prediction outcome, we must bring all features to a comparable scale.

Example code for standardization:

```r
#' Standardize Features for Predictive Modeling
#'
#' @param data A data frame containing features to standardize
#' @param features Character vector of feature names to standardize
#' @param suffix Character suffix for new columns (defaults to "_std")
#' @return Original data frame with added standardized columns
#'
standardize_features <- function(data, features, suffix = "_std") {

 # Calculate means and sds for all features at once
 scaling_params <- data %>%
   summarise(across(all_of(features),
                    list(mean = ~mean(., na.rm = TRUE),
                         sd = ~sd(., na.rm = TRUE)))) %>%
   pivot_longer(everything(),
                names_to = c("feature", "stat"),
                names_sep = "_") %>%
   pivot_wider(names_from = stat, values_from = value)

 # Add standardized versions
 result_data <- data %>%
   mutate(across(all_of(features),
                 ~(. - scaling_params$mean[scaling_params$feature == cur_column()]) /
                   scaling_params$sd[scaling_params$feature == cur_column()],
                 .names = "{.col}{suffix}"))

 # Store parameters as attribute
 attr(result_data, "scaling_params") <- scaling_params

 return(result_data)
}

# Example usage:
features_to_scale <- c("age", "balance", "day", "duration", "campaign", "pdays", "previous")
trainDat <- standardize_features(trainDat, features_to_scale)
testDat <- standardize_features(testDat, features_to_scale)
```

Question 5: Which variables did you scale? Describe the scaling you did.

I scaled each of the continuous numeric variables in the dataset, which include: Age, Balance, Day, Duration, Campaign, Pdays, and Previous. Variables balance and duration were right skewed, so I considered using a log transformation prior to scaling. I decided not to, since the balance variable has useful negative data, and the log transformation would not have been able to capture this.

I standardized each of the numeric variables. Essentially what this standardization process is doing is putting all the variables on an equal playing field, so that variables with greater values are not weighed more than others. After standardizing the numeric variables and one-hot encoding the character variables, we are ready to move onto the most creative part of data prep, feature engineering.

(d) Feature engineering

Predictor variables in machine learning are often called “features.” Feature engineering is the process of transforming raw data into meaningful inputs that enhance the performance of predictive models. It involves creating, modifying, or selecting features to better capture the underlying patterns in the data, leveraging both domain expertise and statistical techniques. While some of the earlier data processing steps may overlap with feature engineering, the focus here is on developing new features from raw data—such as combining variables in ways that extract predictive information or generating key interaction terms. This steps takes advantage of domain knowledge and data experience.

```
# Use this chunk for feature engineering. Feel free to add more chunks too.


# Engineering a binary feature to indicate if an account has a negative balance
trainDat <- trainDat %>%
  mutate(negative_balance = ifelse(balance < 0, 1, 0))

testDat <- testDat %>%
  mutate(negative_balance = ifelse(balance < 0, 1, 0))


# Engineering a binary feature to indicate if an account has a balance in the top 20%
balance_p80 <- quantile(trainDat$balance, 0.80)

trainDat <- trainDat %>%
  mutate(high_balance = ifelse(balance > balance_p80, 1, 0))

testDat <- testDat %>%
  mutate(high_balance = ifelse(balance > balance_p80, 1, 0))


#Engineering a binary feature to indicate if a client has been contacted previously
trainDat <- trainDat %>%
  mutate(previously_contacted = ifelse(pdays != -1, 1, 0))

testDat <- testDat %>%
  mutate(previously_contacted = ifelse(pdays != -1, 1, 0))


# Engineering a binary feature to indicate if the last call is in the top 30% longest (duration)
duration_p70 <- quantile(trainDat$duration, 0.70)

trainDat <- trainDat %>%
  mutate(long_call = ifelse(duration > duration_p70, 1, 0))

testDat <- testDat %>%
  mutate(long_call = ifelse(duration > duration_p70, 1, 0))
```

Question 6: Did you create any new features from your raw data? If so, describe at least one and explain why you thought it might help prediction.

Yes, I created four new features from the raw data. Two of them are in relation to the balance variable. I created a binary indicator for when an observation has a negative balance, and a binary indicator for when an observation has a balance in the top 20%. I decided to do this because of the real world context and domain knowledge. Chances are accounts with the highest balances are more likely to open accounts, and those with the lowest are the opposite. I wanted to try and grab that predictive info with these 2 indicators.

The other two new features I created relate to contact with the customers. First, I built an indicator to show if a client has been previously contacted. The variable pdays has a unique value of -1 for when a customer has never been contacted. I used this to build the binary indicator. The last new feature I created is a flag for long calls, specifically those in the top 30% duration. This is another time where the inspiration comes from domain knowledge. A longer marketing call typically indicates more interest and engagement, which can be extremely useful for predicting the target.

Step 4: Review and check your results

Now that you've finished preparing your training and testing data, it's important to **review your datasets carefully** before moving on to modeling. This step helps you confirm that your processing was successful and that you didn't accidentally introduce problems.

🛠 Example code to help with checks (add more so that you feel confident)

```
### 🛠 Example code to help with checks

# Peek at the first rows
head(trainDat)
```

	age <int>	balance <int>	day <int>	duration <int>	campaign <int>	pdays <int>	previous <int>	y <chr>	y_binary <dbl>
1	58	2143	5	261	1	-1	0	no	0
2	44	29	5	151	1	-1	0	no	0
3	33	2	5	76	1	-1	0	no	0
4	33	1	5	198	1	-1	0	no	0
5	28	447	5	217	1	-1	0	no	0
6	58	121	5	50	1	-1	0	no	0

6 rows | 1-10 of 65 columns

```
# Skim again and focus on new variables
skimr::skim(trainDat)
```

Data summary

Name	trainDat
Number of rows	31647
Number of columns	64
_____	
Column type frequency:	
character	1
numeric	63
_____	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
y	0	1	2	3	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
age	0	1	40.88	10.57	18.00	33.00	39.00	48.00	95.00	▅▇▅▁
balance	0	1	1356.47	3039.70	-8019.00	72.00	448.00	1417.00	102127.00	▇▁▁▁
day	0	1	15.81	8.31	1.00	8.00	16.00	21.00	31.00	▅▅▇▅▅
duration	0	1	257.95	256.99	0.00	103.00	180.00	319.00	4918.00	▇▁▁▁

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
campaign	0	1	2.76	3.08	1.00	1.00	2.00	3.00	63.00	
pdays	0	1	40.39	100.21	-1.00	-1.00	-1.00	-1.00	871.00	
previous	0	1	0.57	1.90	0.00	0.00	0.00	0.00	55.00	
y_binary	0	1	0.12	0.32	0.00	0.00	0.00	0.00	1.00	
job_admin.	0	1	0.11	0.32	0.00	0.00	0.00	0.00	1.00	
job_blue-collar	0	1	0.22	0.41	0.00	0.00	0.00	0.00	1.00	
job_entrepreneur	0	1	0.03	0.18	0.00	0.00	0.00	0.00	1.00	
job_housemaid	0	1	0.03	0.16	0.00	0.00	0.00	0.00	1.00	
job_management	0	1	0.21	0.41	0.00	0.00	0.00	0.00	1.00	
job_retired	0	1	0.05	0.22	0.00	0.00	0.00	0.00	1.00	
job_self-employed	0	1	0.03	0.18	0.00	0.00	0.00	0.00	1.00	
job_services	0	1	0.09	0.29	0.00	0.00	0.00	0.00	1.00	
job_student	0	1	0.02	0.14	0.00	0.00	0.00	0.00	1.00	
job_technician	0	1	0.17	0.37	0.00	0.00	0.00	0.00	1.00	
job_unemployed	0	1	0.03	0.17	0.00	0.00	0.00	0.00	1.00	
job_unknown	0	1	0.01	0.08	0.00	0.00	0.00	0.00	1.00	
marital_divorced	0	1	0.12	0.32	0.00	0.00	0.00	0.00	1.00	
marital_married	0	1	0.60	0.49	0.00	0.00	1.00	1.00	1.00	
marital_single	0	1	0.28	0.45	0.00	0.00	0.00	1.00	1.00	
education_primary	0	1	0.15	0.36	0.00	0.00	0.00	0.00	1.00	
education_secondary	0	1	0.51	0.50	0.00	0.00	1.00	1.00	1.00	
education_tertiary	0	1	0.29	0.46	0.00	0.00	0.00	1.00	1.00	
education_unknown	0	1	0.04	0.20	0.00	0.00	0.00	0.00	1.00	
default_no	0	1	0.98	0.14	0.00	1.00	1.00	1.00	1.00	
default_yes	0	1	0.02	0.14	0.00	0.00	0.00	0.00	1.00	
housing_no	0	1	0.44	0.50	0.00	0.00	0.00	1.00	1.00	
housing_yes	0	1	0.56	0.50	0.00	0.00	1.00	1.00	1.00	
loan_no	0	1	0.84	0.37	0.00	1.00	1.00	1.00	1.00	
loan_yes	0	1	0.16	0.37	0.00	0.00	0.00	0.00	1.00	
contact_cellular	0	1	0.65	0.48	0.00	0.00	1.00	1.00	1.00	
contact_telephone	0	1	0.06	0.25	0.00	0.00	0.00	0.00	1.00	
contact_unknown	0	1	0.29	0.45	0.00	0.00	0.00	1.00	1.00	
month_apr	0	1	0.06	0.25	0.00	0.00	0.00	0.00	1.00	
month_aug	0	1	0.14	0.34	0.00	0.00	0.00	0.00	1.00	
month_dec	0	1	0.00	0.07	0.00	0.00	0.00	0.00	1.00	

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
month_feb	0	1	0.06	0.24	0.00	0.00	0.00	0.00	1.00	
month_jan	0	1	0.03	0.17	0.00	0.00	0.00	0.00	1.00	
month_jul	0	1	0.15	0.36	0.00	0.00	0.00	0.00	1.00	
month_jun	0	1	0.12	0.32	0.00	0.00	0.00	0.00	1.00	
month_mar	0	1	0.01	0.10	0.00	0.00	0.00	0.00	1.00	
month_may	0	1	0.30	0.46	0.00	0.00	0.00	1.00	1.00	
month_nov	0	1	0.09	0.28	0.00	0.00	0.00	0.00	1.00	
month_oct	0	1	0.02	0.13	0.00	0.00	0.00	0.00	1.00	
month_sep	0	1	0.01	0.11	0.00	0.00	0.00	0.00	1.00	
poutcome_failure	0	1	0.11	0.31	0.00	0.00	0.00	0.00	1.00	
poutcome_other	0	1	0.04	0.20	0.00	0.00	0.00	0.00	1.00	
poutcome_success	0	1	0.03	0.18	0.00	0.00	0.00	0.00	1.00	
poutcome_unknown	0	1	0.82	0.39	0.00	1.00	1.00	1.00	1.00	
age_std	0	1	0.00	1.00	-2.16	-0.75	-0.18	0.67	5.12	
balance_std	0	1	0.00	1.00	-3.08	-0.42	-0.30	0.02	33.15	
day_std	0	1	0.00	1.00	-1.78	-0.94	0.02	0.62	1.83	
duration_std	0	1	0.00	1.00	-1.00	-0.60	-0.30	0.24	18.13	
campaign_std	0	1	0.00	1.00	-0.57	-0.57	-0.25	0.08	19.57	
pdays_std	0	1	0.00	1.00	-0.41	-0.41	-0.41	-0.41	8.29	
previous_std	0	1	0.00	1.00	-0.30	-0.30	-0.30	-0.30	28.59	
negative_balance	0	1	0.08	0.28	0.00	0.00	0.00	0.00	1.00	
high_balance	0	1	0.20	0.40	0.00	0.00	0.00	0.00	1.00	
previously_contacted	0	1	0.18	0.39	0.00	0.00	0.00	0.00	1.00	
long_call	0	1	0.30	0.46	0.00	0.00	0.00	1.00	1.00	

```
# Check distributions for each engineered feature
trainDat %>%
  count(negative_balance) %>%
  mutate(percent = n / sum(n) * 100)
```

negative_balance <dbl>	n <int>	percent <dbl>
0	28990	91.604259
1	2657	8.395741

2 rows

```
trainDat %>%
  count(high_balance) %>%
  mutate(percent = n / sum(n) * 100)
```

high_balance <dbl>	n <int>	percent <dbl>
0	25318	80.00126
1	6329	19.99874

2 rows

```
trainDat %>%
  count(previously_contacted) %>%
  mutate(percent = n / sum(n) * 100)
```

previously_contacted <dbl>	n <int>	percent <dbl>
0	25844	81.66335
1	5803	18.33665

2 rows

```
trainDat %>%
  count(long_call) %>%
  mutate(percent = n / sum(n) * 100)
```

long_call <dbl>	n <int>	percent <dbl>
0	22181	70.08879
1	9466	29.91121

2 rows

```
# Alignment check: make sure train and test have identical variables
all.equal(names(trainDat), names(testDat))
```

```
## [1] TRUE
```

If you see mismatches, **fix them before moving on to modeling**.

Question 7: Do you feel confident your data is ready for at least a first round of modeling? Why or why not?

Yes, I feel confident that my data is ready. I worked through each step making sure to avoid leaking data, and thinking of the project as a whole throughout. Because of this, I think I extracted as much useful predictive information as I could have.

Also, I have confirmed my training and testing splits match, and that all the variables I standardized were standardized correctly. The new features I created are all working as intended as well. My data is ready for the first round of modeling.

Step 5: Save your processed training and testing files

```r
# Save you training and testing data files using the file path for dataInputPath
saveRDS(trainDat, file = paste0(params$my_path, "trainDat.rds"))
saveRDS(testDat, file = paste0(params$my_path, "testDat.rds"))
```